

Progetto Gestione di Reti:

Mappatura di rete WI-FI sulla base della distanza da un nodo monitor

Daniele Comaschi

d.comaschi@studenti.unipi.it

Mat. 678922

Scopo:

Il progetto riguarda la creazione di un programma python che in modo passivo, ascoltando il traffico di rete, classifichi i dispositivi della zona wi-fi percepita sulla base della distanza approssimativa di questi dal nodo monitor, ed estragga statistiche rilevanti sulla composizione e il traffico di ogni cluster.

Il programma fornisce anche statistiche globali sulla rete che possono essere confrontate con quelle di ogni cluster.

Scelte architetturali:

La scelta di python come linguaggio per sviluppare il progetto è motivata principalmente dalla libreria [Scapy](#) che offre un parsing automatico dei frame 802.11 tramite delle classi dedicate, semplificando molto l'estrazione delle informazioni dai pacchetti di rete catturati.

Inoltre, la libreria [Pcap-ng](#), essendo fondamentalmente un binding python per la libreria di cattura di pacchetti libpcap, offre gli stessi metodi fondamentali per ascoltare il traffico e catturare i pacchetti mantenendo prestazioni comparabili alla libreria C.

Python però pone anche seri limiti alle prestazioni ottenibili con un'architettura multithread, in quanto il GIL (Global Interpreter Lock) rende impossibile il parallelismo: solo un thread alla volta può acquisire la lock sull'interprete ed eseguire le istruzioni.

Per questa ragione si è scelta un'architettura multiprocesso in modo che la cattura, l'analisi e la rappresentazione dei dati possano procedere in parallelo, senza race condition sull'interprete.

Il programma è quindi costituito da:

- Uno Sniffer per la cattura dei pacchetti
- Un'Analyzer per l'analisi dei pacchetti catturati
- Una GUI (minima)

La comunicazione tra i processi, in particolare quella tra Sniffer ed Analyzer e quella tra Analyzer e GUI, avviene tramite due code multiprocesso.

Il programma contiene anche delle funzioni per la gestione automatizzata delle interfacce di rete (modulo `ifaces_mgmt.py`).

Sniffer:

E' implementato nel modulo `sensor.py`.

Per ottimizzare la fase di cattura, è stato scelto di separare totalmente quest'ultima da quella di analisi, evitando qualsiasi preprocessing dei pacchetti.

Lo sniffer si limita quindi a catturare i pacchetti tramite `pcapy-ng` ascoltando il traffico su tutti i canali wi-fi tramite channel-hopping e inviando i pacchetti catturati al processo Analyzer tramite un'apposita coda.

L'invio dei pacchetti sulla coda comporta la loro serializzazione (e relativa deserializzazione da parte dell'Analyzer), quindi si è scelto un'invio bufferizzato, che anziché spedire pacchetto per pacchetto, spedisca un batch di pacchetti alla volta, quando il numero di pacchetti catturati raggiunge un determinato "SEND THRESHOLD".

Determinare tale valore, nonché andare a definire un sistema di comunicazione tra Sniffer ed Analyzer che risultasse mediamente efficiente nel tempo, indipendentemente dalle variazioni

improvvisi del traffico di rete, è stata indubbiamente una delle principali sfide nella realizzazione del progetto.

Dopo pochi test è subito emersa la necessità di un sistema adattivo, che reagisse ai picchi e ai relativi cali improvvisi del traffico di rete. Per ottenere ciò si sono esplorate diverse possibilità, quali:

- **send_threshold = pkts /sec medi catturati durante la sessione:** questo metodo, per quanto estremamente semplice, ha riscontrato grosse problematiche in termini di reattività. Uno (e non l'unico) dei casi in cui fallisce è il seguente:
 - Si ha una fase prolungata di traffico elevato => send_threshold si staziona su un valore abbastanza alto.
 - Si ha un calo improvviso del traffico => send_threshold cambia di poco, in quanto la media dei pkts/sec è "bloccata" su un valore alto da molte misurazioni precedenti.
 - Il programma perde di reattività perché il send_threshold viene raggiunto molto lentamente, quindi i pacchetti vengono inviati all'analyzer con una frequenza molto bassa.
- **send_threshold = (pkts/sec)*buffering_factor:**
questo metodo consiste nel far comunicare lo Sniffer e l'Analyzer con un buffering_factor gestito da quest'ultimo.
Se l'Analyzer rileva un carico molto basso (dimensione della coda dei pacchetti vicina allo zero), diminuisce il buffering factor, facendo inviare allo sniffer i pacchetti più frequentemente.
Se invece rileva un carico elevato (dimensione della coda dei pacchetti elevata), aumenta il buffering factor, facendo inviare i pacchetti allo sniffer con una frequenza minore.
Dai test effettuati, il metodo è risultato corretto nel meccanismo, ma non efficace:
Nel passaggio da una situazione di traffico elevato a una di basso traffico, il progressivo decremento del buffering factor faceva effettivamente riprendere la reattività al programma, ma solo dopo il tempo necessario alla stabilizzazione del buffering factor su un nuovo valore di equilibrio.
Un comportamento speculare lo si otteneva nel passaggio da una situazione di basso traffico ad una di traffico elevato.
In pratica, il problema fondamentale di questo approccio è che tenta di curare una situazione indesiderata solo dopo che questa si è verificata, invece che adattarsi immediatamente per prevenirla.
- **send_threshold = EMA (pkt catturati):** questo metodo è quello che ha dato di gran lunga i risultati migliori ed è quindi quello adottato nel progetto.
Il send threshold è uguale alla media mobile esponenziale dei pacchetti catturati, e viene ricalcolato ogni volta che si catturano dei pacchetti da un canale.
Si utilizza la EMA con uno smoothing factor molto alto (0.8), in modo da dare molto peso agli ultimi valori registrati e poco a quelli vecchi, per garantire reattività.
Questo metodo ha mostrato nei test di mantenere una dimensione della coda dei pacchetti tendente allo zero. Inoltre permette di disaccoppiare la reattività del programma dalle prestazioni dello sniffer, in quanto un send threshold che cambia continuamente adattandosi immediatamente al traffico di rete, garantisce che lo Sniffer invii sempre pacchetti sulla coda.
Di seguito i risultati di un test:
 - BASSO TRAFFICO (catturati 3 pkts su un canale) => EMA = 3, send_threshold = 3
 - BURST DI TRAFFICO (catturati 1270 pkts su un canale)
=> $EMA = 1270 * (0.8) + 3 * (0.2) = 1016,6$
=> il send threshold diventa immediatamente pari a 1016 pkt, quindi viene fatto un solo invio di 1016 pacchetti nella coda, che non esplode di dimensione.
 - CALO IMMEDIATO DEL TRAFFICO (catturati 30 e 20 pkts su due canali)
=> $EMA = 30 * (0.8) + 1016 * (0.2) = 227,2$
=> $EMA = 20 * (0.8) + 227,2 * (0.2) = 61,4$

=> send_threshold torna ad un valore molto basso nel giro di due sole catture, mantenendo la reattività.

In pratica, questo metodo sembra riuscire a catturare la natura “spiky” del traffico wireless.

Oltre a questo, lo sniffer presenta due piccole ottimizzazioni:

- I canali wi-fi, dopo esser stati ascoltati tutti almeno una volta, vengono ordinati in ordine decrescente rispetto al traffico rilevato su ognuno di essi, e i canali in cui non è stato trovato nessun pacchetto vengono momentaneamente scartati, per poi essere testati nuovamente in modo sporadico.
- Se lo sniffer rileva un burst di traffico, il send threshold non viene aggiornato nella cattura dei pacchetti sul canale successivo.
Questo dà tempo all’analyzer di elaborare la grande quantità di pacchetti ricevuta a seguito del burst, prima di ottenere nuovamente dati da lavorare.

Analyzer:

E’ implementato nel modulo analyzer.py.

Riceve i pacchetti catturati dallo sniffer e li processa, estraendone le informazioni utili e istanziando e popolando le strutture dati da inviare all’interfaccia.

L’Analyzer esegue un ciclo infinito di analisi dei pacchetti e ogni volta che viene superato un certo intervallo di tempo (REFRESH TIME, fissato a 2 sec.), aggiorna le statistiche con i dati ottenuti e le invia all’interfaccia tramite l’apposita coda.

L’Analyzer calcola tre principali tipologie di statistiche:

- Quelle di Performance, che offrono informazioni sulle prestazioni del programma e permettono di individuare eventuali situazioni di sovraccarico.
- Quelle di Traffico, che offrono informazioni sul traffico rilevato.
- Quelle dei Clusters, che offrono informazioni riguardo al traffico e alla composizione di ogni cluster.

Per ognuna di queste tipologie, l’Analyzer calcola statistiche sia di sessione che di “round”:

- **Le statistiche di sessione** sono calcolate con i dati ottenuti dall’inizio dell’esecuzione del programma.
Queste sono utili per ottenere una panoramica generale sull’andamento della sessione di cattura, ma con il passare del tempo diventano poco sensibili agli ultimi dati catturati.
- **Le statistiche di “round”** sono calcolate con i dati ottenuti nell’ultimo round di analisi.
Fissato il refresh_time a 2 sec., un round di analisi corrisponde a circa 2 sec. dedicati interamente all’analisi dei pacchetti ricevuti, quindi le statistiche di round si basano sui dati analizzati negli ultimi due secondi.
Queste statistiche sono efficaci nel catturare variazioni istantanee dei valori considerati, come picchi di traffico, variazione improvvisa nella composizione del traffico e sovraccarico.

Una volta calcolate le statistiche, l’Analyzer le aggrega in una struttura dati unica e invia quest’ultima alla GUI.

GUI:

E’ implementata nel modulo gui.py.

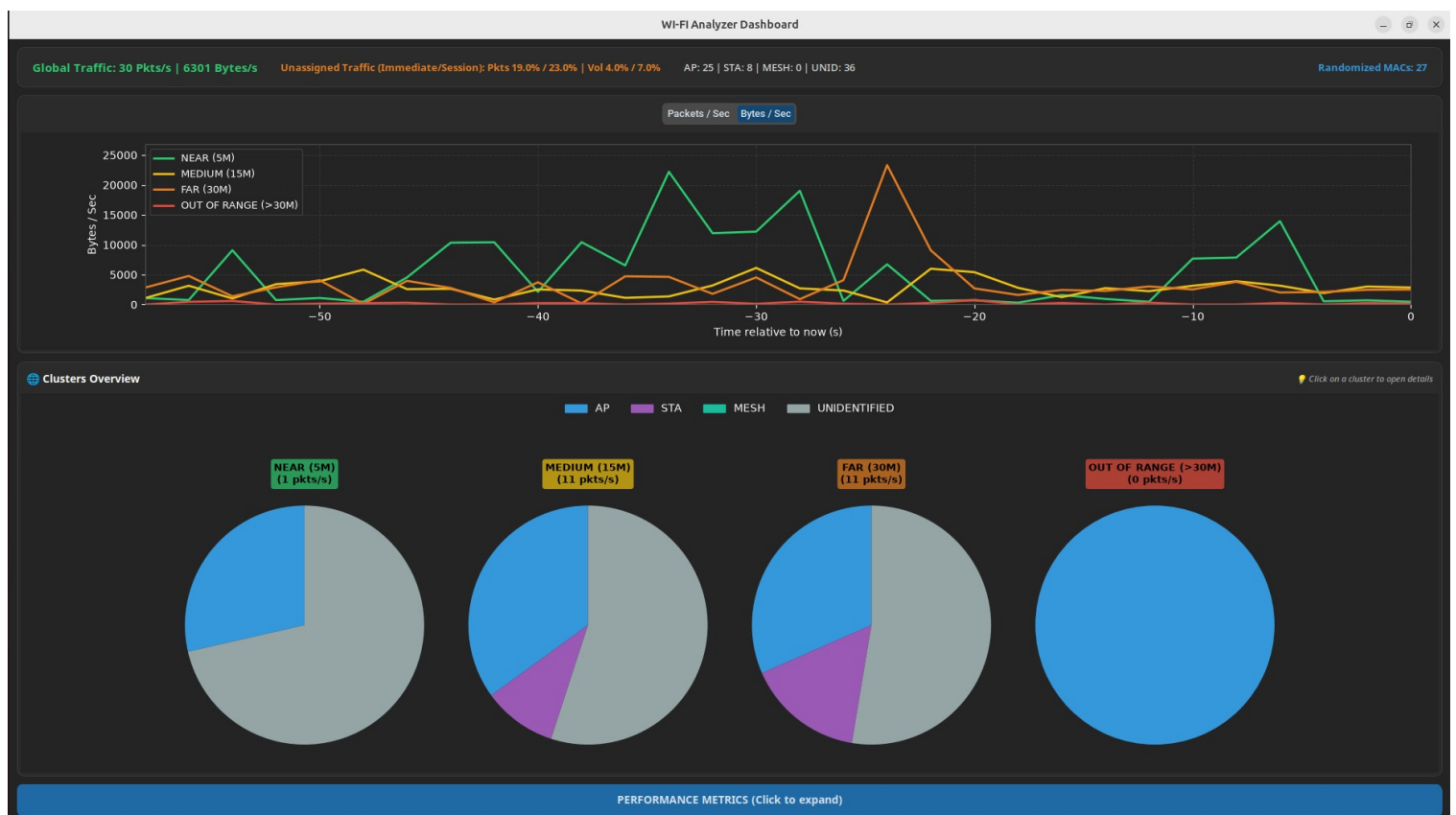
L’interfaccia grafica riceve i dati inviati dall’analyzer, e li rappresenta in un’unica finestra interattiva, in costante aggiornamento.

L’interfaccia, basata su [matplotlib](#) e [customtkinter](#), è minimal ed è stata generata tramite IA.

Avendo la necessità di visualizzare i dati in modo compatto, ma non avendo competenze riguardo la creazione di interfacce (solo una conoscenza generica di cosa sia un interface loop e di come funzioni customtkinter), mi sono esclusivamente limitato a disegnarla e definirne l’interazione con

il processo Analyzer.

Di seguito, la dashboard principale dell'interfaccia:



Classificazione in base alla distanza:

I dispositivi individuati vengono classificati sulla base della distanza calcolata dal nodo monitor, tramite il valore RSSI (Received Signal Strength Indicator) associato ai loro frame 802.11.

Questo è possibile grazie al [log distance path loss model](#), che esprime la potenza del segnale ricevuto in funzione della distanza, legandoli tramite una relazione logaritmica.

$$\text{RSSI}(d) = \text{RSS}(d_0) - 10n \log(d/d_0) + X_0$$

Il modello può essere semplificato (ed è stato usato in modo semplificato), evitando di considerare X_0 (ovvero una v. a. gaussiana che rappresenta il rumore), e fissando d_0 (la distanza di riferimento) ad 1m, ottenendo:

$$\text{RSSI}(d) = \text{RSS}_{\text{ref}} - 10n \log(d)$$

Si inverte quindi la formula per ottenere la distanza a partire dall'RSSI ricevuto, ottenendo:

$$d = 10^{((\text{RSSI}(d) - \text{RSSI}_{\text{ref}})/(-10n))}$$

Per poter utilizzare la formula è quindi necessario stabilire un RSSI_{ref} (RSSI di riferimento ad 1m di distanza) ed un path loss exponent (n) che determina di quanto il segnale si degradi all'aumentare della distanza.

I limiti nell'accuratezza e nell'affidabilità delle misurazioni posti dalla scelta di questi due parametri sono trattati nella sezione "Limiti e possibili soluzioni".

Una volta calcolata la distanza stimata tramite la formula sopra, si osserva in quale dei 4 intervalli prestabiliti il dispositivo ricade:

- Near: il dispositivo si trova nel raggio di 5m
- Medium: il dispositivo si trova tra i 5 e 15m di distanza
- Far: il dispositivo si trova tra i 15 e i 30m di distanza
- Out of Range: il dispositivo si trova oltre i 30m di distanza

Ottenuto quindi il cluster in cui il dispositivo ricade secondo tale misurazione, non gli si assegna direttamente tale cluster, ma si assegna un “voto” verso il cluster specificato. Questo permette di ottenere un partizionamento più affidabile, basato su diverse misurazioni nel tempo, cercando di minimizzare l’effetto di misurazioni rumorose o incoerenti. Un esempio è il seguente:

MAC ADDRESS	CLUSTERS
‘AB:CD:EF:01:02:03’	NEAR: 232 MEDIUM: 87 FAR: 6

All’indirizzo mac (che nel programma identifica un dispositivo) sono associate varie misurazioni (1 misurazione = 1 pacchetto ricevuto da tale indirizzo):

232 lo classificano come NEAR, 87 come MEDIUM e 6 come FAR.

A tale dispositivo sarà quindi associato il cluster NEAR.

Test effettuati:

Oltre ai test intermedi eseguiti per capire quale fosse il modo migliore di gestire il traffico di rete, sono stati effettuati dei test in live con la versione finale del progetto:

- Test a basso traffico: il programma è stato testato in una zona caratterizzata da basso/medio traffico (ambiente domestico) ed ha sostenuto varie sessioni di cattura (1, 5, 7 minuti) senza alcun problema rilevato.
La dimensione media della coda dei pacchetti è rimasta tendente allo zero durante tutto il tempo di esecuzione.
- Test con traffico medio/alto: il programma è stato testato con un traffico mediamente elevato (in media 700pkts/sec) in una sessione di 7 minuti.
Anche in questo caso la dimensione media della coda è rimasta tendente allo zero.
Il test è avvenuto sempre in ambiente domestico, attivando continuamente degli [speed test](#) con altri dispositivi, andando a simulare un continuo flusso elevato di pacchetti.
- Test con burst di traffico: il programma è stato testato con burst di traffico elevati (1200 pkts/sec) alternati a momenti di traffico medio/basso.
Il programma si è dimostrato reattivo ai picchi e ai cali di traffico improvvisi.

Dai test sono però emersi dei limiti fondamentali riguardo l’affidabilità della mappa di rete costruita dal programma che sono trattati nella successiva sezione.

Limiti e possibili soluzioni:

- **Randomizzazione dei MAC address**: la problematica principale riscontrata dai test è data dal fatto che molti dispositivi, talvolta anche AP, utilizzano la randomizzazione dei MAC address.
Dato che nel programma un dispositivo è identificato univocamente (e solamente) da un indirizzo MAC, questo provoca un aumento incontrollato dei “dispositivi” rilevati nel tempo, rendendo affidabile la mappa costruita solamente durante i primissimi minuti di esecuzione del programma.
Inoltre, diversamente da quanto dichiarato nella issue di github, non ho calcolato statistiche

come numero medio di pacchetti per dispositivo o volume medio di traffico per dispositivo: proprio statistiche del genere, che si basano sulla quantità di dispositivi rilevati, verrebbero gradualmente diluite nel numero crescente di dispositivi fittizi, risultando in indicatori statistici fuorvianti e incompatibili con le misure di traffico globale.

Ho preferito quindi concentrarmi su statistiche che non sono direttamente o fortemente influenzate da questo problema (ovviamente, rimango disponibile ad implementare il calcolo delle statistiche escluse se lei lo ritenesse necessario).

In futuro, implementare un sistema di de-randomizzazione dei MAC address aumenterebbe significativamente l'affidabilità della mappa costruita e sarebbe fondamentale nel prevenire la degradazione di questa nel tempo.

- **Alto numero di dispositivi non identificati:** un'altra problematica evidente riscontrata dai test è quella dell'elevato numero di dispositivi che risultano UNIDENTIFIED (una delle quattro classi di dispositivi possibili tra AP, STA, MESH, UNIDENTIFIED).

In genere, circa la metà dei dispositivi totali rilevati è classificato come non identificato. Questo avviene quasi sicuramente a causa della semplicità dell'euristica utilizzata per stabilire il tipo di dispositivo:

- Un dispositivo è AP se il suo indirizzo compare come Transmitter Address di un Beacon Frame.

- Un dispositivo è STA se il suo indirizzo compare come Transmitter Address di una Probe Request.

- Un dispositivo è MESH se per un dato indirizzo si verificano entrambe le condizioni precedenti.

In pratica, per l'identificazione di un dispositivo si utilizzano solo due tipologie di Frame di Management (Beacon e Probe Request) che risultano limitate nello svelare l'identità di quest'ultimo.

In futuro si potrebbe pensare di utilizzare una gamma più vasta di frame per svelare l'identità dei dispositivi che attualmente vengono classificati come non identificati.

- **Affidabilità nella misura delle distanze:** come precedentemente discusso, per poter utilizzare il log distance path loss model è necessario fissare un RSSI di riferimento e un path loss exponent, che avranno un impatto determinante nella computazione delle distanze. Prima di testare il progetto, si sono stabiliti questi due valori nel seguente modo:
 - $RSSI_{ref}$ è stato calcolato ponendo il dispositivo a 1m di distanza in linea d'aria da un'access point e calcolando l'RSSI medio dei frame ricevuti da tale access point.
 - il path loss exponent è stato calcolato ponendo il dispositivo a varie distanze (1m, 5m, 10m (approssimativamente)) dallo stesso access point, ottenendo quindi i rispettivi RSSI dei frame ricevuti e isolando il path loss exponent nella formula.

I valori così calcolati sono $RSSI_{ref} = -34\text{dBm}$ e $path_loss_exponent = 4.5$.

Con questi valori, ottenuti in ambiente chiuso con presenza di mura, un RSSI pari a -90dbm (valore basso) viene associato a una distanza di circa 17.5m, mentre un RSSI pari a -70 dBm (valore medio) viene associato ad una distanza di circa 6.4m.

Le distanze sembrerebbero quindi abbastanza realistiche (o quantomeno accettabili) per ambienti chiusi.

Purtroppo però, la precisione di queste misurazioni, essendo fortemente legata a questi due parametri, può facilmente variare e diventare irrealistica, ad esempio se:

- Il dispositivo su cui si esegue il programma ha una scheda wireless più o meno sensibile rispetto a quella utilizzata per la calibrazione.

- Si esegue il programma in un ambiente diverso da quello in cui sono stati ottenuti i due parametri.

Per risolvere questo problema si potrebbe implementare una fase di calibrazione all'avvio del programma, tenendo conto però che questa richiederebbe un'estesa collaborazione da

parte dell'utente, che deve porre il dispositivo su cui sta eseguendo il programma a diverse distanze da un dispositivo di riferimento per un certo intervallo di tempo, per poter ottenere i dati necessari da cui stabilire i due parametri.

Istruzioni per l'utilizzo:

Per implementare una gestione automatizzata delle interfacce, si è utilizzato la libreria pyroute2, che scambia messaggi con il kernel per modificare o creare interfacce di rete.

Inoltre, per poter eseguire il programma evitando che questo erediti i permessi da super user, si è utilizzato la libreria pyprctl.

Per questi motivi, il programma funziona **esclusivamente su linux**.

Per l'avvio occorre:

- Aprire una bash linux e spostarsi all'interno della cartella del progetto
- Creare un virtual environment python nella cartella del progetto, con all'interno:
 - pyprctl (per la gestione delle capabilities)
 - pyroute2
 - pcap-ng
 - scapy
 - customtkinter
 - matplotlib
- Rimuovere dal venv python qualsiasi symlink all'interprete python3 e copiare un interprete python3 all'interno del venv.
- Dare all'interprete python3 i permessi cap_net_admin e cap_net_raw con:
"sudo setcap cap_net_admin,cap_net_raw=eip nome_venv/bin/python3"
- Attivare il venv python ("source nome_venv/bin/activate")
- Lanciare il programma dalla shell con "python3 main.py"

Per interrompere il programma, è sufficiente chiudere la finestra della GUI, oppure si può utilizzare Ctrl + C da terminale.

In entrambi i casi, l'interfaccia di rete originale viene automaticamente ripristinata e portata nella modalità managed.